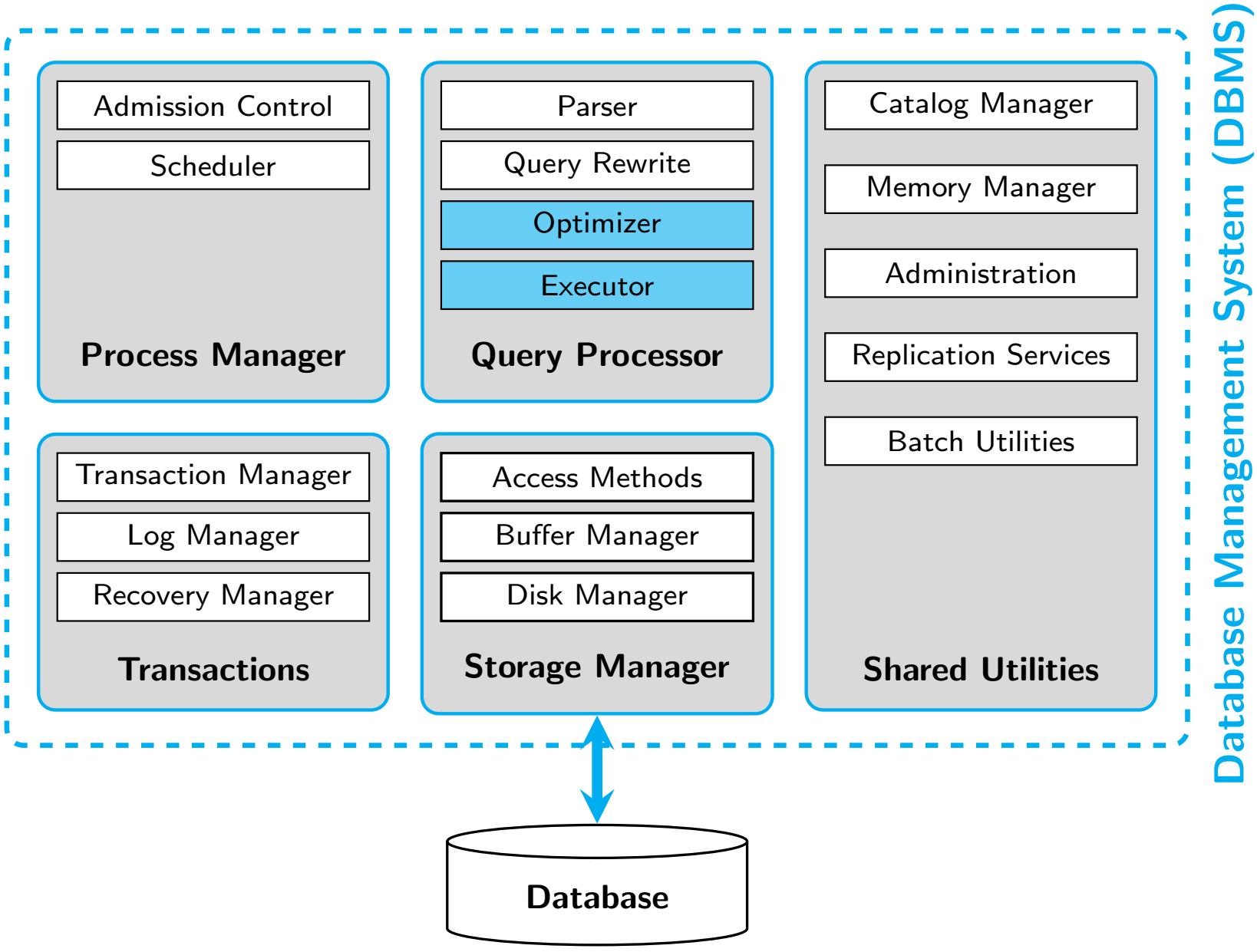
Optimización del plan físico

Clase 20

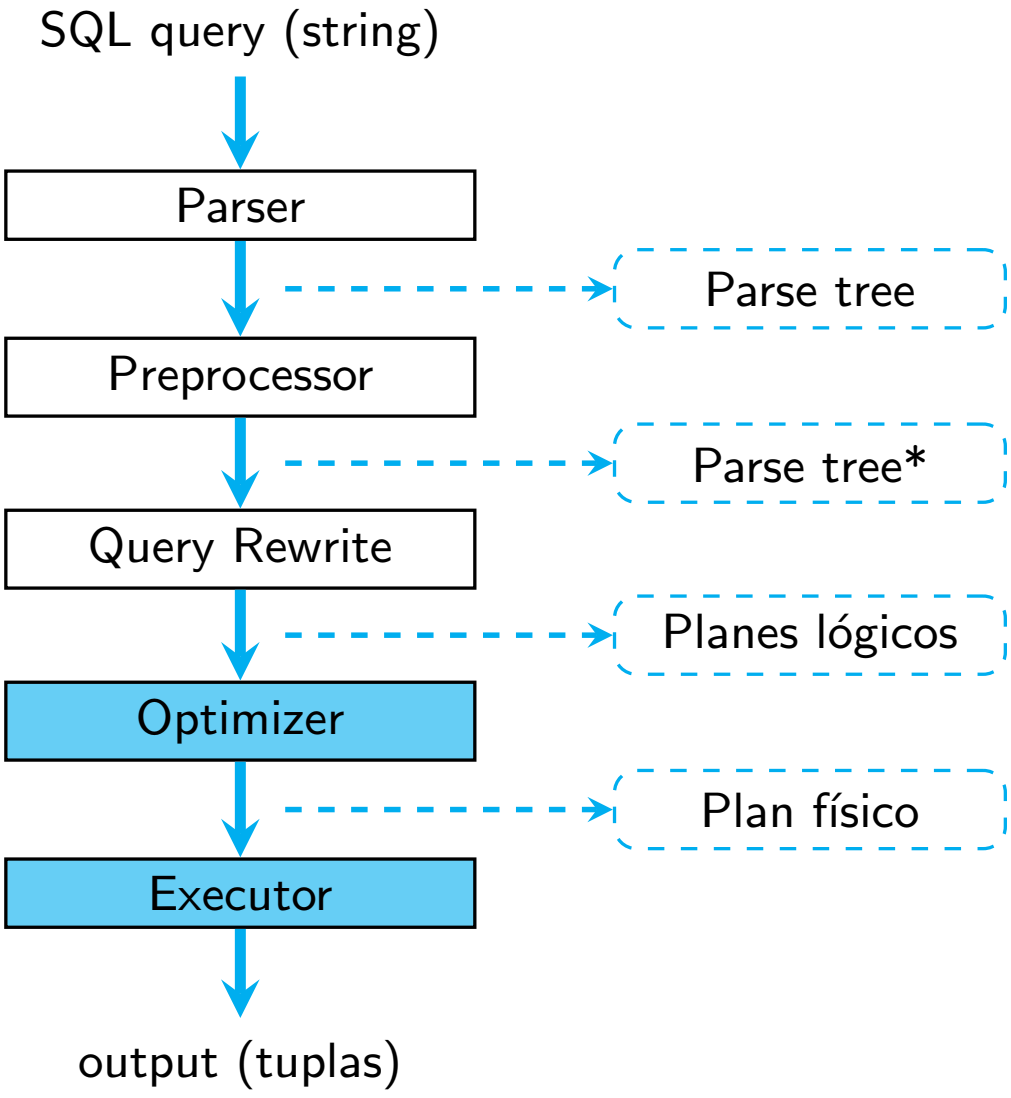
IIC2413 - Sección 3

Prof. Cristian Riveros

Optimización de consultas



Zoom al optimizador de consultas



Outline

Join (anterior)

Busquedad de planes físicos

Orden de los joins

Materialización

Outline

Join (anterior)

Busquedad de planes físicos

Orden de los joins

Materialización

Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join ✓
3. Index Nested Loops Join ✓
4. Sort-Merge Join
5. Hash Join

Joins $\bowtie_{A=B}$: Sort-Merge Join

People

name	idc
Juan	1
Pedro	2
Diego	2
Andrés	2
Bastián	4

$\bowtie_{\mathbf{idc=id}}$

Course

id	name
1	Base de datos
2	Programación avanzada
3	Estructura de datos

(solo cierto para **equijoins**)

¿cuál es la diferencia entre **merge** y **join**?

Joins $\bowtie_{A=B}$: Sort-Merge Join

Algoritmo

input: Operadores R y S, un predicado $A = B$.

```
open()
    R' := merge-sort(R)
    S' := merge-sort(S)
    r := R'.next()
    s := S'.next()
    fillBuffer()

close()
    R'.close()
    S'.close()
```

```
fillBuffer()
    while r(A) ≠ s(B) do
        if r < s then
            r := R'.next()
        else
            s := S'.next()
    Buff := ∅
    while r(A) = s(B) do
        Buff := Buff ∪ {s}
        s := S'.next()
```

Joins $\bowtie_{A=B}$: Sort-Merge Join

Algoritmo

input: Operadores R y S, un predicado $A = B$.

```
next()
    while r ≠ NULL do
        t := Buff.next()
        if t = NULL then
            r' := r
            r := R'.next()
            if r(A) = r'(A) then
                Buff.reset()
            else
                fillBuffer()
        else
            return (r, s)
    return NULL
```

Joins $\bowtie_{A=B}$: Sort-Merge Join

Costo Sort-Merge Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) = \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) = 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

¿es más **eficiente**?

Joins $\bowtie_{A=B}$: Sort-Merge Join

Ejemplo

- R y S son tablas de 16 MB.
- Cada página tiene 8 KB.
- Cada relación tiene 2.048 páginas.

Si consideramos que cada I/O toma $\approx$ 0,1 ms (mínimo):

1,2 segundos

...1/3 menos que BNLJ! :)

Conclusiones sobre Sort-Merge Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) \quad = \quad \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) \quad = \quad 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

- Operador físico preferido para $\bowtie_{A=B}$ (sin suposiciones).
- Más rápido aún si ambas relaciones ya están **ordenadas**.

¿cuándo podría suceder que **ambas relaciones estén ordenadas**?

(B+tree)

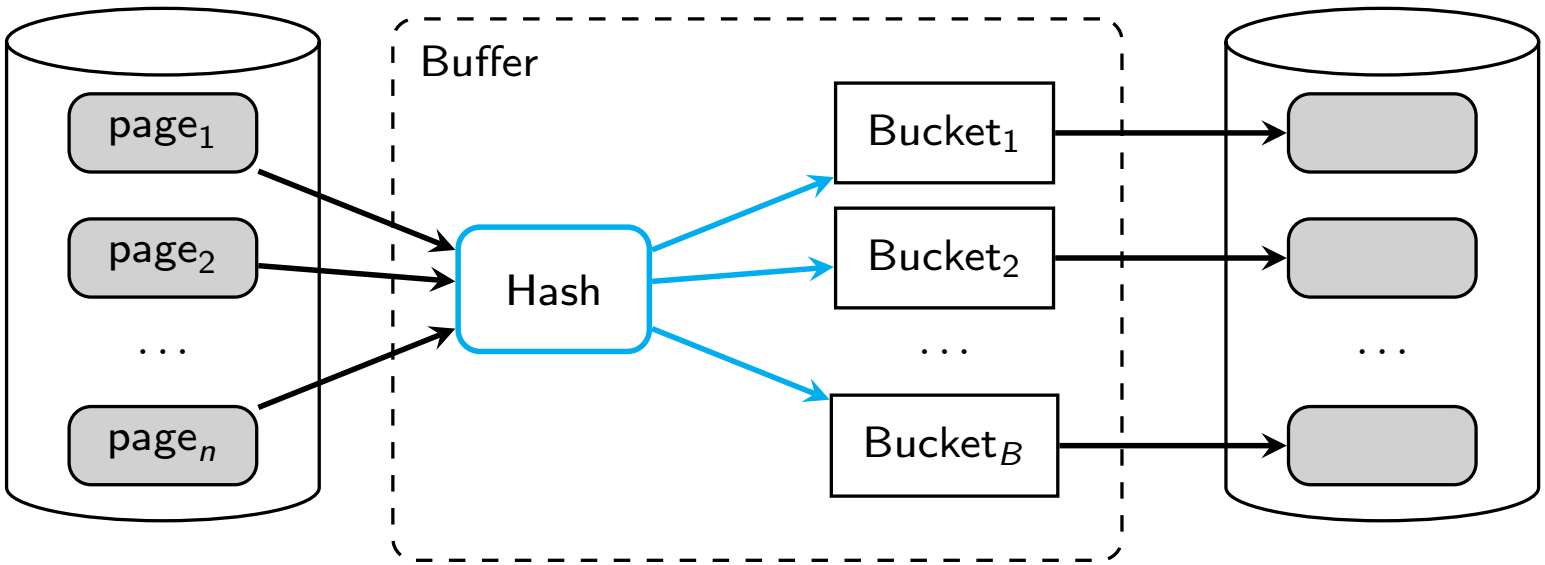
Joins físicos para $R_1 \bowtie_\theta R_2$

1. Nested Loops Join ✓
2. Block Nested Loops Join ✓
3. Index Nested Loops Join ✓
4. Sort-Merge Join ✓
5. Hash Join

Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición



Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición

   - Cada página P del operador R se lee a memoria.
   - Por cada $t \in P$ se computa $h(t)$ y se envía al bucket $R_{h(t)}$.
   - Cada página P del operador S se lee a memoria.
   - Por cada $t \in P$ se computa $h(t)$ y se envía al bucket $S_{h(t)}$.

     si un **bucket esta completo**, se vacía y materializa en disco.

Joins $\bowtie_{A=B}$: Hash Join

Algoritmo en dos fases

1. Fase de partición
2. Fase de join de inter-particiones
   - Join $R_i \bowtie_{A=B} S_i$ entre los bucket R_i y S_i, recursivamente.
   - (Idealmente) R_i o S_i sea almacenado completamente en el buffer.

¿cuántos buckets son suficientes para tener **solo una fase** de partición?

Joins $\bowtie_{A=B}$: Hash Join

Costo Hash Join

Formula general

$$\text{cost}(R \bowtie_{A=B} S) = \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

Relaciones

$$\text{cost}(R \bowtie_{A=B} S) = 3 \cdot (\text{pages}(R) + \text{pages}(S))$$

Mismo costo que **Sort-Merge Join**

Sort-Merge Join versus Hash Join

$$\text{cost}(R \bowtie_{A=B} S) \;=\; \text{cost}(R) + \text{cost}(S) + 2 \cdot (\text{pages}(R) + \text{pages}(S))$$

¿cuál es la **diferencia** entre ambos operadores físicos?

- Sort-merge join funciona con cualquier **distribución** de los datos.
- El resultado de sort-merge join es **ordenado**.
- Hash Join permite sacarle mayor provecho al **buffer**.

Outline

Join (anterior)

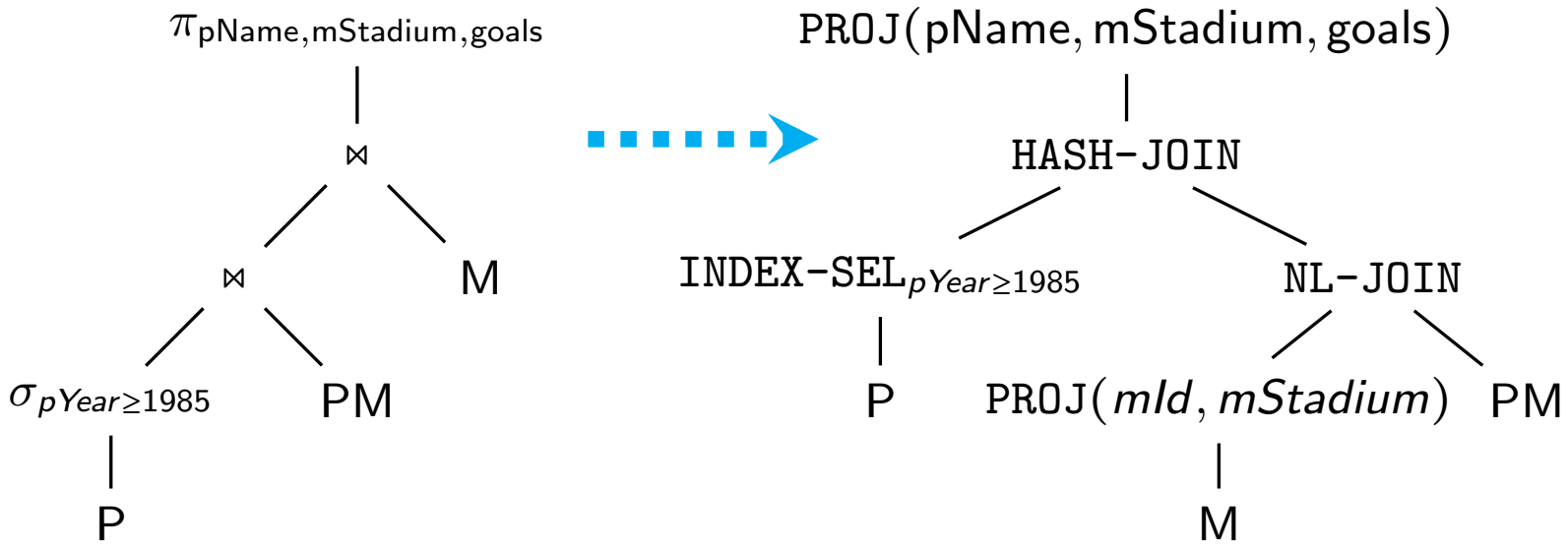
Busquedad de planes físicos

Orden de los joins

Materialización

¿cómo escogemos el plan físico?

1. Convertir los planes lógicos en planes físicos.
2. Calcular el **costo** de cada plan físico.
3. Escoger el plan con menor costo.



¿cómo **calculamos** ese costo?

Estimación de costos y cardinalidades

Para escoger el mejor **plan físico** debemos estimar cual es su **costo total**.

- costo de uno o varios **operadores físicos**.
- tamaño de resultados intermedios.
- selectividad de un predicado.
- cantidad de valores distintos.

"Performance del DBMS esta fuertemente ligado a una buena estimación del costo de sus planes."

Estimación de costos y cardinalidades

Parámetros dependientes de los operadores físicos

cost(R):	costo (en I/O) para computar R.
pages(R):	número de páginas necesarias para almacenar R.
$\|R\|$:	número de tuplas en R.
tsize(R):	tamaño promedio de una tupla en R.

Parámetros dependientes de los datos

$\text{sel}_\theta(R)$:	$\|\sigma_\theta(R)\| / \|R\|$
distinct(R):	cantidad de elementos distintos en R.
$\text{distinct}_A(R)$:	cantidad de elementos distintos en el atributo A.

Veremos algunos **ejemplos** y **técnicas comunes** de como estimarlos

Estimación del output de una selección $\sigma_{A=c}(R)$

Suposición S1: uniformidad de duplicados

"La cantidad de valores duplicados N en $\pi_A(R)$ se distribuye uniformemente sobre los valores distintos en $\pi_A(R)$."

$$\text{sel}^*_{A=c}(R) \;=\; \frac{N}{N \cdot \text{distinct}_A(R)} \;=\; \frac{1}{\text{distinct}_A(R)}$$

Estimación del **tamaño del output** para $\sigma_{A=c}(R)$:

$$|\sigma_{A=c}(R)|^* \;=\; \text{sel}^*_{A=c}(R) \cdot |R| \;=\; \frac{|R|}{\text{distinct}_A(R)}$$

¿es $\text{sel}^*_{A=c}(R)$ una buena estimación? ¿que tan mala puede ser?

Estimación del output de una conjunción: θ_1 AND θ_2

Suposición S2: independencia de predicados

"La selectividad de cada subpredicado es independiente del resto."

$$\text{sel}^*_{\theta_1 \text{ AND } \theta_2}(R) = \text{sel}^*_{\theta_1}(R) \cdot \text{sel}^*_{\theta_2}(R)$$

¿cuál es la estimación del output de $\sigma_{A=10 \text{ AND } B=20}(R)$?

Consideremos una relación $R(A, B)$ con $|R| = 10.000$ tal que:

$$\begin{aligned} \text{distinct}_A(R) &= 50 \\ \text{distinct}_B(R) &= 100 \end{aligned}$$

$$\text{sel}^*_{A=10}(R) = \frac{1}{50} \qquad \text{sel}^*_{B=20}(R) = \frac{1}{100} \qquad \text{sel}^*_{A=10 \text{ AND } B=20}(R) = \frac{1}{5000}$$

$$|\sigma_{A=10 \text{ AND } B=20}(R)|^* = \text{sel}^*_{A=10 \text{ AND } B=20}(R) \cdot |R| = \frac{1}{5000} \cdot 10000 = 2$$

Estimación del output de un join: $R(X, Y) \bowtie S(Y, Z)$

Suposición S3: contención de conjunto de valores

"Si Y es una atributo que aparece en varias relaciones, entonces cada relación escoge sus valores desde el comienzo de una lista fija de valores $a_1, a_2, a_3, \ldots$ y tiene todos los valores de este prefijo."

¿qué relaciones satisfacen la suposición?

- $R(X, Y) = \{(a, 1), (a, 2), (b, 2), (c, 1)\}$
  $S(Y, Z) = \{(2, x), (3, y), (1, z), (4, x)\}$
- $R(X, Y) = \{(a, 4), (a, 3), (b, 7), (c, 9)\}$
  $S(Y, Z) = \{(7, x), (2, y), (3, z), (9, x)\}$

Estimación del output de un join: $R(X, Y) \bowtie S(Y, Z)$

Suposición S3: contención de conjunto de valores

> *"Si Y es una atributo que aparece en varias relaciones, entonces cada relación escoge sus valores desde el comienzo de una lista fija de valores $a_1, a_2, a_3, \ldots$ y tiene todos los valores de este prefijo."*

Consecuencia de S3

Si $\text{distinct}_Y(R) \leq \text{distinct}_Y(S)$, entonces:

$$\delta(\pi_Y(R)) \subseteq \delta(\pi_Y(S))$$

¿és **realista** esta suposición?

- claves foráneas.
- joins con pocos valores "sin match".

Estimación del output de un join: $R(X, Y) \bowtie S(Y, Z)$

Asumiendo S1 y S3

- si $\text{distinct}_Y(R) \leq \text{distinct}_Y(S)$ entonces:

$$\begin{aligned}
\text{sel}^*_{R.Y=S.Y}(R \times S) &= \frac{|\sigma_{R.Y=S.Y}(R \times S)|}{|R \times S|} \\
&= \frac{\sum_{a \in \pi_Y(R)} |\sigma_{Y=a}(S)|}{|R| \cdot |S|} \quad (S3) \\
&= \frac{\sum_{a \in \pi_Y(R)} \frac{|S|}{\text{distinct}_Y(S)}}{|R| \cdot |S|} \quad (S1) \\
&= \frac{1}{\text{distinct}_Y(S)}
\end{aligned}$$

- si $\text{distinct}_Y(R) \geq \text{distinct}_Y(S)$ entonces:

$$\text{sel}^*_{R.Y=S.Y}(R \times S) = \frac{1}{\text{distinct}_Y(R)}$$

Estimación del output de un join: $R(X, Y) \bowtie S(Y, Z)$

Asumiendo S1 y S3

- si $\text{distinct}_Y(R) \leq \text{distinct}_Y(S)$ entonces:

$$\text{sel}^*_{R.Y=S.Y}(R \times S) \quad = \quad \frac{1}{\text{distinct}_Y(S)}$$

- si $\text{distinct}_Y(R) \geq \text{distinct}_Y(S)$ entonces:

$$\text{sel}^*_{R.Y=S.Y}(R \times S) \quad = \quad \frac{1}{\text{distinct}_Y(R)}$$

Juntando ambos casos:

$$\text{sel}^*_{R.Y=S.Y}(R \times S) \quad = \quad \frac{1}{\max\{\text{distinct}_Y(R), \text{distinct}_Y(S)\}}$$

Estimación de **tamaño del output** para $R(X, Y) \bowtie S(Y, Z)$:

$$|R(X, Y) \bowtie S(Y, Z)|^* \quad = \quad \frac{|R| \cdot |S|}{\max\{\text{distinct}_Y(R), \text{distinct}_Y(S)\}}$$

Estimación de elementos distintos

Debemos hacer dos tipos de estimaciones:

1. (**Relación**) Dado una relación R, como mantener:

$$\text{distinct}_L(R)$$

2. (**Operador**) Dado las relaciones R y S y un operador $*$, como estimar:

$$\text{distinct}_L(*(R)) \quad o \quad \text{distinct}_L(R * S)$$

donde L es una lista de atributos.

Estimación de elementos distintos para una **relación**

Para una relación R, podemos calcular $\text{distinct}_L(R)$:

1. Periódicamente y almacenarlo en el **catálogo**.
2. En cada **actualización**.

Estimación de elementos distintos para $R \bowtie S$

Suposición S4: conservación del conjunto de valores

> *"Los valores R de un atributo A, que NO participan en un join, se conservan en el resultado."*

Si A no es atributo de S entonces:

$$\text{distinct}_A^*(R \bowtie S) \quad = \quad \text{distinct}_A(R)$$

¿es esta suposición **realista**?

- claves foráneas.

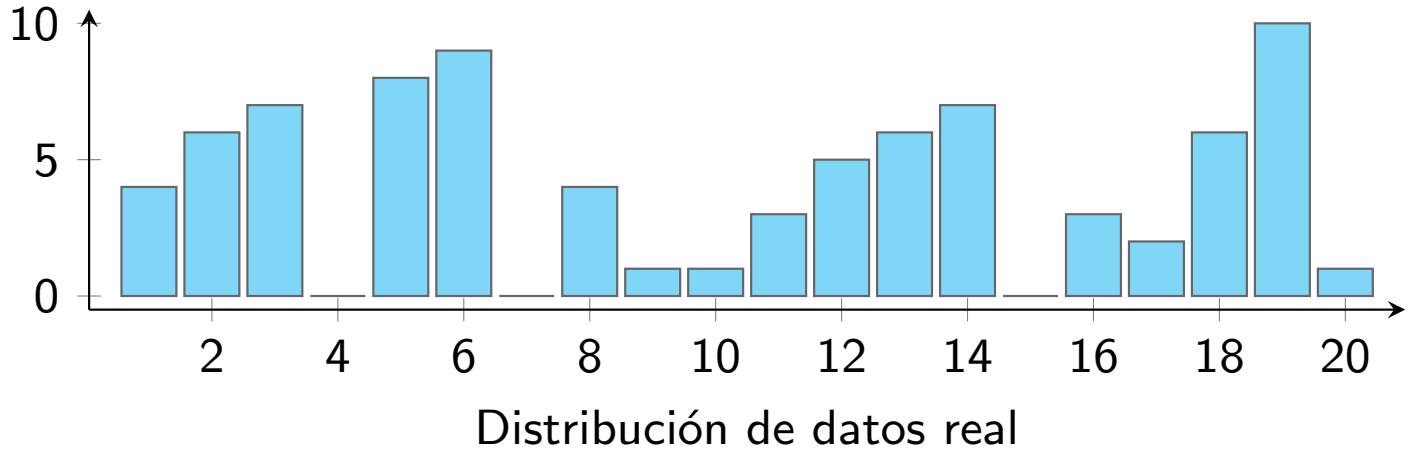
Histogramas

- En la práctica, valores **NO** siguen una distribución uniforme.
- DBMS usan **histogramas** para modelar esta no-uniformidad.
  - Dominio es dividido en intervalos.
  - Por cada intervalo se mantiene:
    1. Frecuencia (cantidad) de elementos.
    2. Cantidad de elementos distintos.

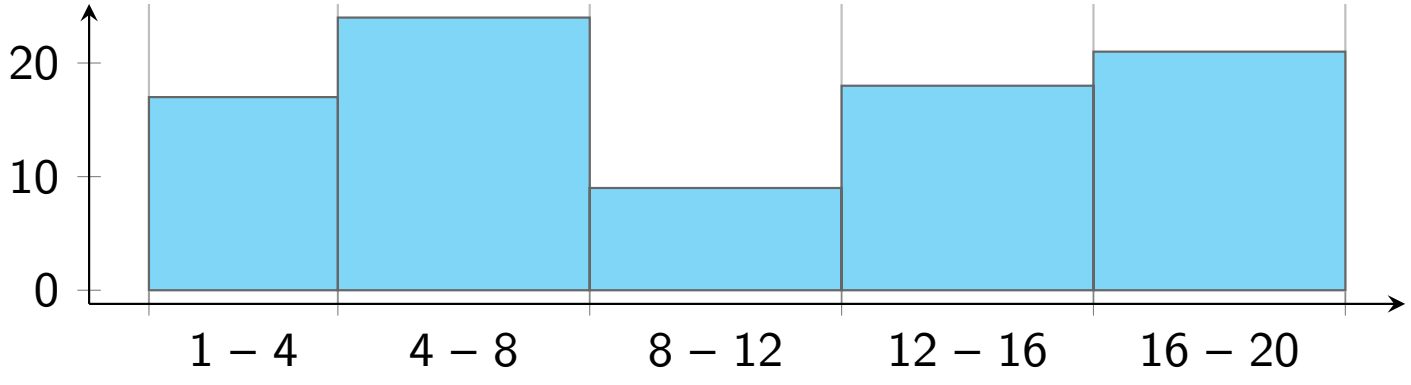
Todo DBMS mantiene histogramas actualizados en su catálogo

Tipos de Histogramas

Histogramas **equi-width**

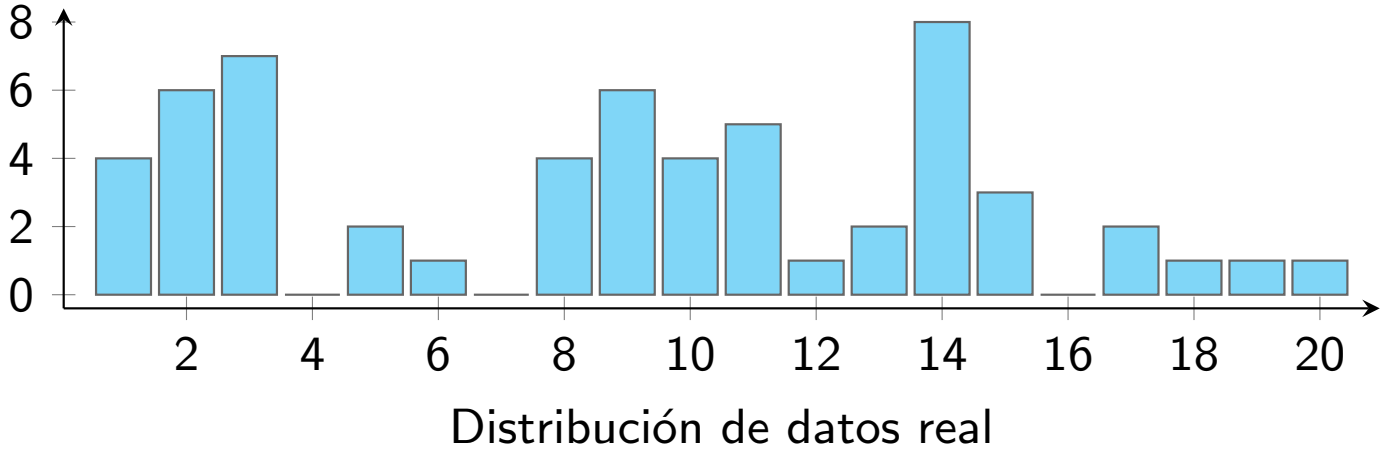


Distribución de datos real

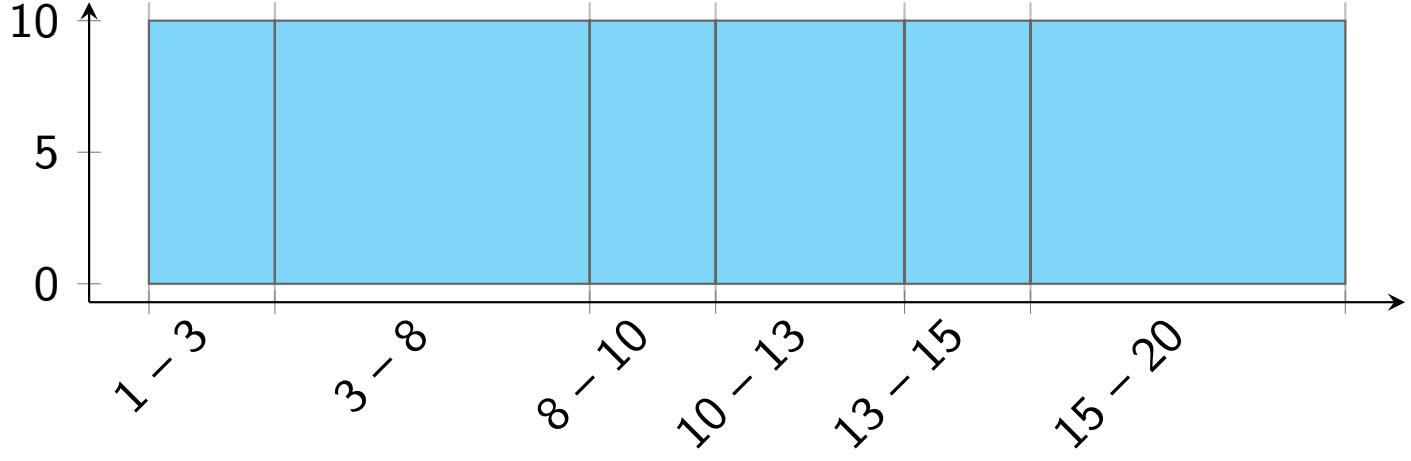


Tipos de Histogramas

Histogramas **equi-depth**



Distribución de datos real



Estadísticas e histogramas son útiles hasta cierto punto

Estadísticas e histogramas:

1. susceptibles a **errores**.
2. solo disponibles para relaciones **bases**.
3. composición de operadores produce
   un aumento **exponencial** del error en la estimación.

"On the Propagation of Errors in the Size of Join Results"
Ioannidis y Christodoulakis, 1993.

Outline

Join (anterior)

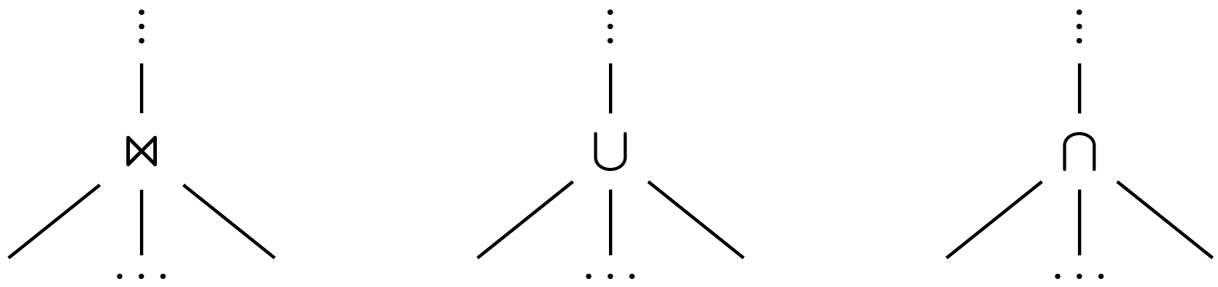
Busquedad de planes físicos

Orden de los joins

Materialización

Orden de operadores conmutativos y asociativos

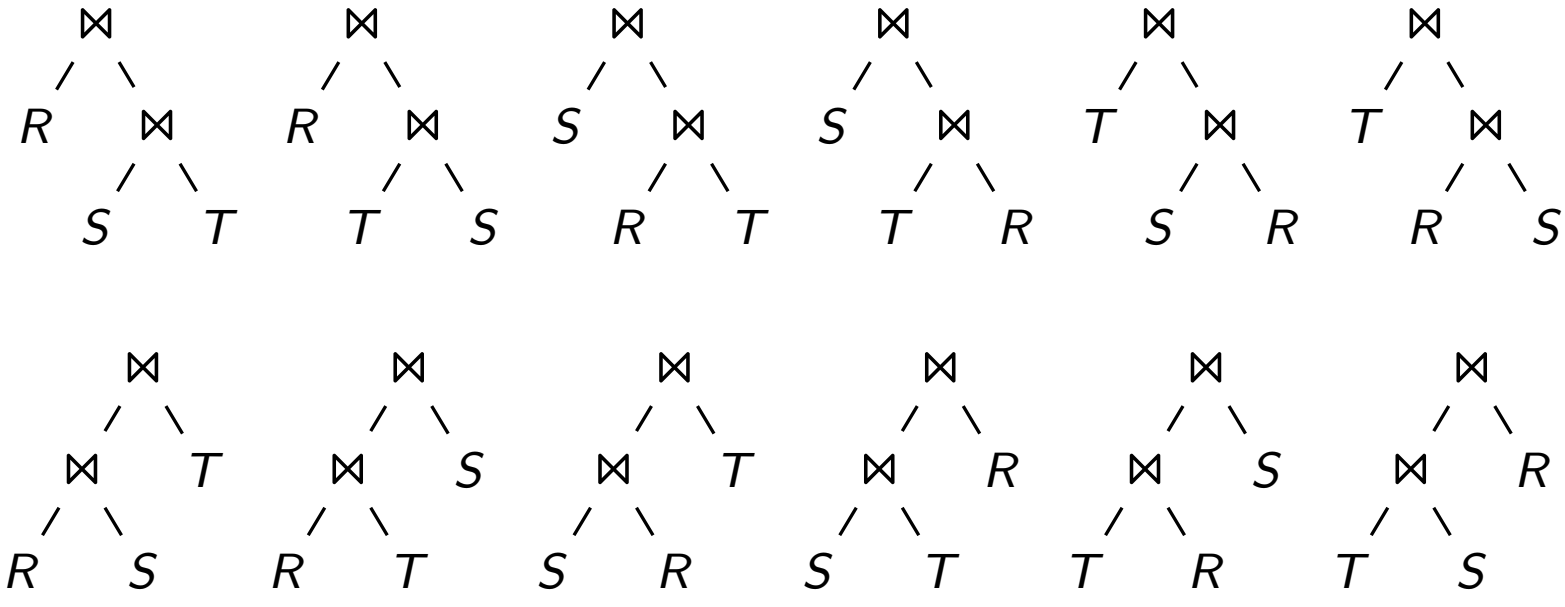
Dado un plan de lógico con nodos de la forma:



¿qué **orden** de los operadores usamos en el plan físico?

Orden de joins

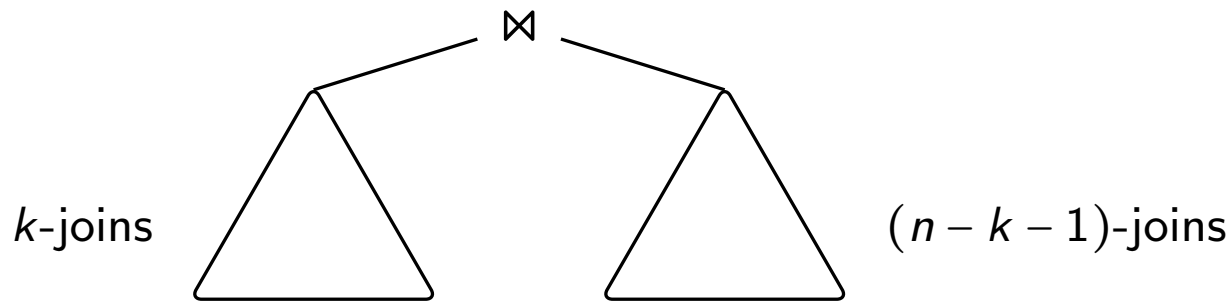
Para $R \bowtie S \bowtie T$, tenemos todos estos **planes** posibles:



¿cuál de todos escogemos?

¿cuántos planes de joins podemos escoger?

Para n-joins de la forma $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_{n+1}$:



El número de posibles árboles de join para n-joins:

$$C_n = \sum_{k=0}^{n-1} C_k \cdot C_{n-k-1}$$

(conocidos como "Catalan numbers")

¿cuántos planes de joins podemos escoger?

Resolviendo esta ecuación recursiva:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

Multiplicando por todas las permutaciones $n!$ de las n relaciones:

$$\#\text{-planes de joins} = \frac{(2n)!}{(n+1)!}$$

Una **cantidad exponencial** de posibles planes!

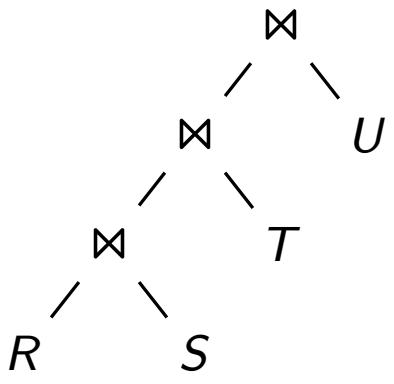
¿y ahora quién podrá defendernos?

Tres opciones para enumerar planes de joins:

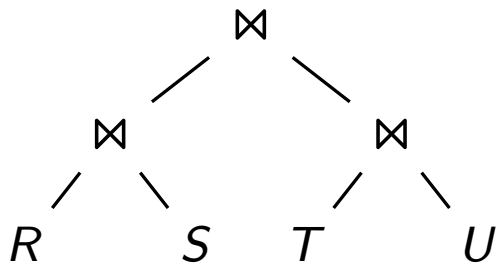
1. Considerarlos todos.
2. Considerar un subconjunto de los planes.
3. Usar una heurística.

2. Considerar un subconjunto de los planes

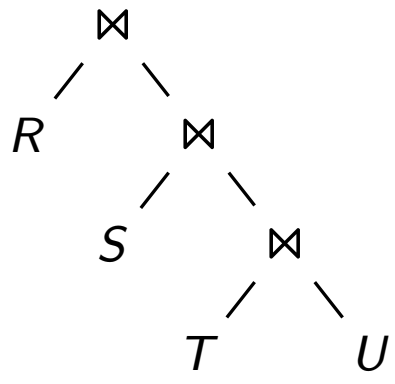
Distintos tipos de planes:



Left-deep plan



Bushy plan



Right-deep plan

¿qué **ventaja** tiene cada plan?

2. Considerar un subconjunto de los planes

Considerar solo **left-deep** plans.

Optimizador de **Selinger**

Ventajas

- Un subconjunto menor de planes a considerar.
- Buena interacción con los operadores físicos de join.

3. Usar una heurística

Heuristica **greedy** para: $R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$

- Mantener un conjunto de planes $\mathcal{P}$.
- Iniciar el conjunto de planes $\mathcal{P}$ con todas las relaciones:

$$\mathcal{P} = \{R_1, \ldots, R_n\}$$

- En cada iteración, escoger dos planes $P_1, P_2 \in \mathcal{P}$ tal que:

$$\text{cost}(P_1 \bowtie P_2) = \min_{P, P' \in \mathcal{P}} \text{cost}(P \bowtie P')$$

y reemplazar $\mathcal{P} := (\mathcal{P} - \{P_1, P_2\}) \cup \{P_1 \bowtie P_2\}$.

¿cuál es el **tiempo** de esta heurística?

Outline

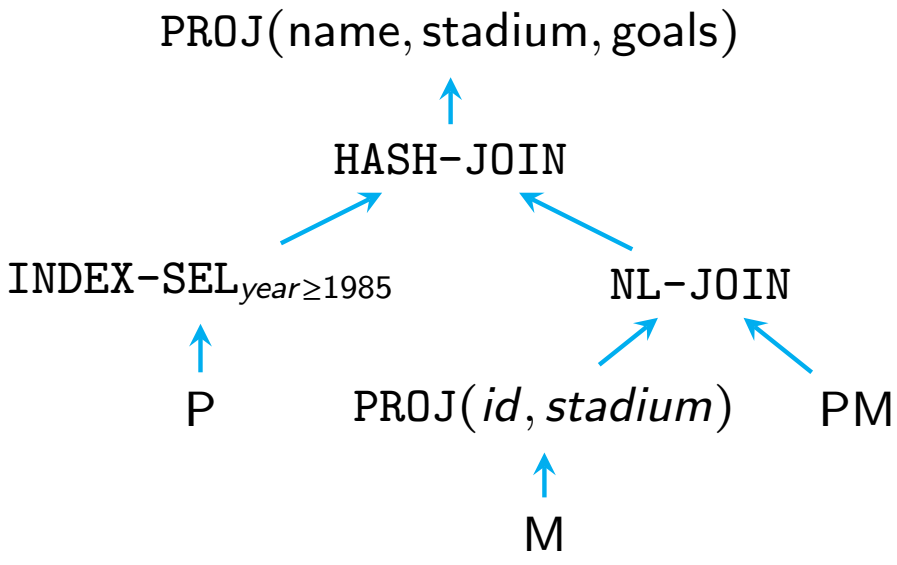
Join (anterior)

Busquedad de planes físicos

Orden de los joins

Materialización

Pipeline



- Ejecución **en serie** del plan físico.
- Cada operador genera **una tupla a la vez**.

Cada operador retorna **uno a uno** sus resultados al operador padre

Bloqueo de operadores

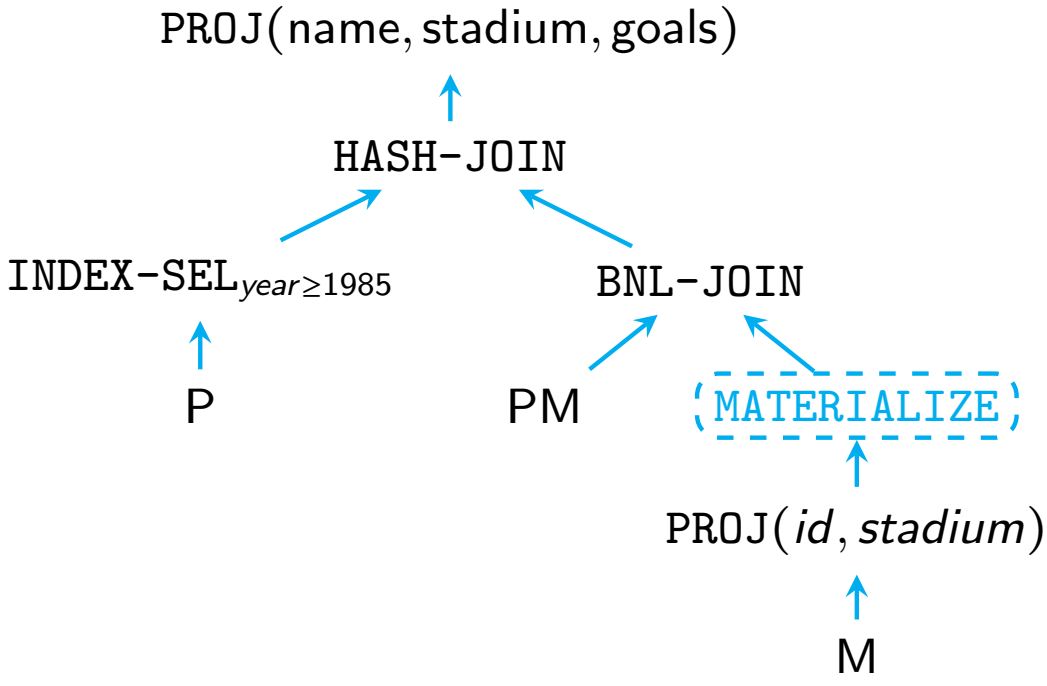
Algunos operadores deben esperar a terminar todo el proceso:

- external-sorting.
- joins.
- group-by y eliminación de duplicados.

Estos operadores usualmente requieren:

materialización (alternativa a Pipeline).

Materialización



- Almacena el resultado de la subconsulta en disco.
- Solo necesario si la subconsulta es requerida reiteradas veces.

Pipeline vs Materialización

Siempre se prefiere pipeline sobre materialización

Usar materialización si:

- Subconsulta es llamada reiteradas veces.
- Costo de almacenamiento es menor a recalcular la consulta.

Finalmente, ejecutamos nuestro plan físico de consultas...

